

Idýl: A Reactive Functional Language for Time, with Csound as a First-Class Backend

Johann PHILIPPE¹

Independant

Abstract. Idýl is a functional, reactive programming language designed to describe and program *time*. Inspired by Antescofo and by Faust’s principle that “everything is a stream”, it treats time as a first-class value and change as the object of computation, rather than as a side effect of a general-purpose runtime. Definitions are stateless by default; state lives explicitly inside temporal blocks driven by a high-accuracy, drift-free scheduler. The language is generalist by intent: composition, performance, live-coding, and the control of arbitrary devices. It reaches the outside world through *modules*, C++ extensions that expose external APIs to Idýl code. Its reference module targets Csound: the scheduler drives one or more independent Csound instances, sequencing i-events, writing control channels, and polling channels back into the reactive graph. We present the language, its temporal semantics, the Csound bridge, and the engine that executes it.

Keywords: language, reactive, functional, time, Csound, live-coding

“The tools we use have a profound (and devious!) influence on our thinking habits, and, therefore, on our thinking abilities.”

— E. W. Dijkstra [1]

1 Introduction

Antescofo, TidalCycles, ChucK [2], SuperCollider [3], and of course Csound [4,5]: these are the languages one thinks of first when programming time in the field of musical creation. Each comes with its own set of qualities and its own paradigm, and each, in turn, shapes how its user is able to think about time [6].

Antescofo, developed at IRCAM by Jean-Louis Giavitto, occupies a specific place in this list. It is the only one that is not *also* an audio programming language. Setting aside its listening machine [7] – which concerns only very specific musical projects – Antescofo’s engine can be described with simplicity: it is an extremely complete musical scheduler [8,9], whose timed reactive language has a formally defined operational semantics [10]. It provides nearly everything a composer may require: time as a first-class value, temporal and pitch-related primitives, and a large collection of built-in functions thought out for computer-music designers. Yet its community remains small: it is closed-source, works mostly inside Max/MSP with no standalone version or API, targets music exclusively, and is a complex language not designed for conciseness.

This locates a gap. On one side stand powerful but effectively unavailable languages such as Antescofo; on the other, live-coding environments such as TidalCycles [11], whose pattern algebra is expressive and immediate but deliberately restricts general programming. Idýl was designed to sit between the two: a language whose central concern is *time*, but which remains a full, open programming language usable well beyond music.

The idea grew over several years and the project began in early 2026. Idýl is a functional, reactive language, fully open-source (GPLv3+), with a simple mechanism to reach the outer world (*modules*) and a concise syntax. Its second inspiration, after Antescofo, is Faust [12,13]: the conviction that *everything is a stream*. In Idýl a constant is a zero-argument function, a signal is a function that changes over time, and a musical sequence is a stream indexed by a clock — an approach to time-varying values whose roots in computer music go back at least to Arctic [14]. The remainder of this paper presents the language (Sect. 2), its Csound module (Sect. 2.3), and its engine (Sect. 3).

2 The Language

2.1 Functions, Flows, and Temporal Blocks

By default, functions in Idyl are stateless and non-temporal. A standard function is written with the usual, minimal syntax; the body *is* the result, and there is no `return`:

```
add(a, b) = a + b
c = add(1, 2)      // computed once
d = add(a, sig)    // if sig is a temporal signal, d re-computes
                  // automatically whenever sig changes state
```

This last line illustrates the core of the language. `add` is a pure function, yet by receiving a temporal argument it is inserted into a *reactive chain*: its result is recomputed each time an input changes. Control flow is functional — there is no `if/else` or `while`; selection is a ternary (`cond ? false; true`), iteration is either a generator expression or an imperative `each` loop.

Time is a first-class value. Literals carry temporal meaning and all normalize to milliseconds internally, so arithmetic between them is natural: `440hz` becomes its period, `120bpm` is 500 ms, `2s + 100ms` is 2100 ms, and `4.0b` counts beats at the current tempo.

Temporal functions carry state across time. The `|>` operator introduces a lambda block with an optional `init` section and an update body; the `dt` parameter sets the tick interval at which the scheduler re-evaluates the block. State is thus explicit and local — the only place mutation is allowed:

```
// a phase accumulator ticking every 10 ms
free_phasor(freq, dt=10ms) = phase |> {
  init: { phase = 0 }
  phase = fmod(phase + dt / freq, 1)
}
```

Flows are the central data structure: ordered streams of values, written as literals anywhere an expression is valid. They admit generator expressions, live temporal elements per slot, and — crucially for rhythm — *trigger gating*. A trigger (!) is an event; a rest (.) is its absence. The `on` keyword advances or fires only on live triggers:

```
flow pattern = {
  rhythm : [!, ., ., !, !, .]
  melody on rhythm : [60, 63, 65]  // cursor advances only on !
}
```

Temporal blocks may also *emit* named side-channel values, read elsewhere with the `::` accessor, and *catch* the events they raise (`catch timer::finished: { ... }`). A one-sample delay operator `'(expr)` (or `'(expr, N)` for N samples) gives per-expression feedback. Together these turn a program into a dataflow graph that the scheduler walks deterministically.

Synchrony, determinism, dataflow. Within one reaction chain, updates are *synchronous*, in the sense of the synchronous languages such as Esterel [15]: on each scheduler tick the chain is re-evaluated top-to-bottom, in source order, as a single atomic step (Fig. 1). Given identical inputs a chain therefore produces identical output — the update rule is *deterministic*, and the only non-determinism is the one the programmer requests explicitly (`rnd`, `dust`). Different temporal instances may tick at different `dt`; they are independent synchronous islands scheduled on a single, shared, drift-free clock, and the engine-time-locked built-ins `phasor(clk, N)` and `sync(clk, N)` align islands that started at different moments. The overall model is thus *reactive dataflow*: a temporal binding followed by reactions that re-run when their inputs change, close in spirit to functional reactive programming [16].

2.2 The Standard Library

Because temporal functions and flows are expressive on their own, most of what a user needs is *written in Idyl itself* rather than baked into the engine. The standard library `stdlib.idyl` defines oscillators, trigger utilities (`metro`, `euclid`, `dust`), envelopes, and mapping helpers as ordinary Idyl definitions; `music.idyl` adds scales and microtonal *temperament* tables, returning decimal MIDI notes whose fractional part encodes cent deviations — directly consumable by Csound's `cpsmidinn`; `chaos.idyl` collects nonlinear maps for generative material.

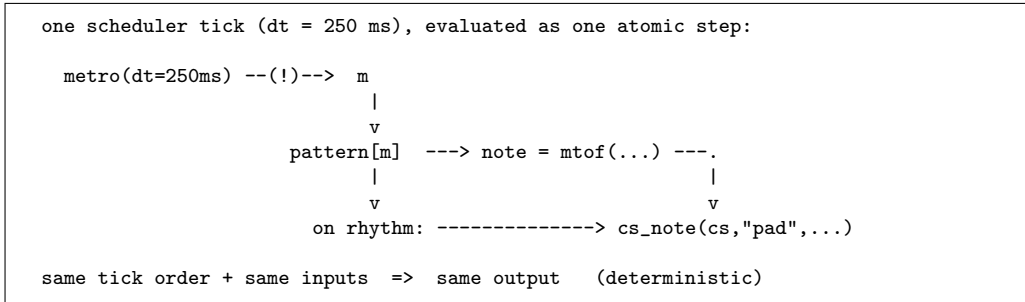


Fig. 1. A reaction chain. A clock trigger enters at the top; the flow is indexed, mapped to a frequency, and gated before reaching Csound. The whole chain is one synchronous, deterministic step.

2.3 Modules: Reaching the Outside World, with Csound First

A *module* is not an Idýl library but a compiled C++ extension: it wraps an external API and exposes a set of functions to Idýl code — the door through which Idýl controls the world. A built-in OSC module is provided and external modules load as shared objects; but the reference module, and the reason this work is presented at a Csound conference, is the Csound bridge.

Each `cs_open()` call compiles a `.csd` or `.orc` file and starts an *independent* Csound instance in its own performance thread, so several instances may run at once. The bridge is bidirectional (Fig. 2): Idýl schedules i-events with `cs_note`, writes software-bus channels with `cs_chnset`, and — through the native temporal `cs_chnget` — *reads* channels back into the reactive graph, where a Csound-side value can drive Idýl decisions.

The idiom is compact. A clock drives a flow; the flow is indexed and gated; the gated events become Csound notes:

```

module("csound")
import("stdlib")  import("music")

cs = cs_open("pad.csd")

flow phrase = {
  rhythm : [!, _, !, !, _, !, _, _]
  deg    on rhythm : [0, 2, 4, 3, 5]
}

process play: {
  m = metro(dt = 250ms)          // clock -> trigger stream
  s = phrase[m]                  // index the flow on every tick
  on s.rhythm: {                  // fire only on live triggers
    f = tune(c4, minor, s.deg, just) // microtonal frequency
    cs_note(cs, "pad", 400ms, 0.2, f) // i "pad" 0 0.4 0.2 f
  }
}

```

Continuous control is equally direct — an Idýl LFO becomes a Csound control channel, and a channel written by Csound can be polled back:

```

process control: {
  lfo_val = lfo(0.3hz, 0.5, dt = 20ms) // Idyl side
  cs_chnset(cs, "cutoff", 800 + lfo_val * 600) // write to Csound

  env = cs_chnget(cs, "rms", dt = 20ms) // read back (temporal)
  catch env::changed: { print("csound rms:", env) }
}

```

Here Idýl is neither merely a score generator nor merely a controller: its scheduler and Csound's audio engine form one reactive system, each able to observe the other. Because `cs_note` preserves fractional `p1` values, simultaneous calls to the same base instrument spawn independent voices rather than retriggering one — polyphony falls out of the flow model for free.

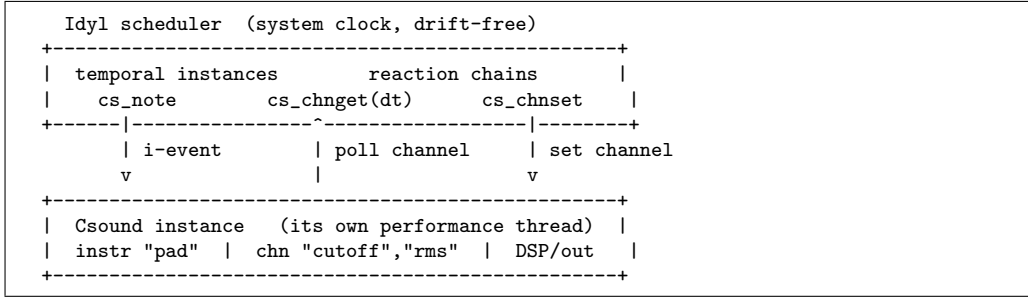


Fig. 2. The bidirectional Csound bridge. One `cs_open` instance runs its own `csoundPerformKsmps` loop; Idyl sends i-events and channel writes, and reads channels back as native temporal signals.

3 The Engine

Idyl source is compiled through a Flex/Bison front-end into a typed AST with per-node location tracking, then validated by a multi-pass semantic analyzer (symbol collection and import resolution, name resolution, arity and argument checking, soft type inference, temporal validation, flow validation, and scope checking). Every diagnostic is reported *before* a single line runs.

Execution then splits along two paths. Pure user functions and reaction lists are compiled to a compact stack-based *bytecode* and run on a small expression VM; temporal functions, flow operations, and control-flow constructs the compiler does not cover fall back to a tree-walking interpreter, two classic implementation strategies [17]. Temporal functions are instantiated with isolated state and subscribed to the scheduler.

Time management is the heart of the system. The scheduler is a high-accuracy, drift-free, system-clock loop on its own thread; each temporal subscription is a periodic callback whose next deadline is computed from an absolute origin rather than accumulated increments, so no error builds up over long performances. Process blocks — the only executable entry points — create the reaction chains of Fig. 1. Clocks form a hierarchy in which a tempo change propagates proportionally to children, and the `phasor/sync` built-ins expose engine time directly for phase-accurate coordination.

Timing accuracy. We measured this claim externally [18]: a program sending one OSC message every 10 ms ran for 180 s while a separate process kernel-timestamped each arrival. With `--system-clock`, Idyl drifts by $+0.002 \pm 0.001$ ppm (about $36 \mu\text{s}$ per hour), on par with SuperCollider’s clocks [3], with a jitter of $\sigma = 5.7 \mu\text{s}$ (98 % of intervals within $\pm 15 \mu\text{s}$). Csound [4] and ChuckK [2] show millisecond jitter, since their messages leave once per audio buffer; ChuckK, paced by the sound card, also drifts by -16.6 ppm. The default audio-clock mode drifts by -1 ppm, a nanosecond-rounding artefact of its kernel timer.

Live-coding and validation. In `--listen` mode the engine loads a file without running it and waits for OSC control. Sending a running process block to `/idyl/eval diffs` it against the live one: surviving temporal instances keep their state, while changed reactions and `dt` values apply on the next tick, in the tradition of just-in-time programming [19]. Driven from a VS Code or Vim plugin, this hot-reload makes Idyl usable on stage; it was tested at a live-coding concert in a French national creation centre in June 2026, with Csound as the sound engine.

Implementation note. Idyl follows a design-first, AI-assisted workflow: the author specifies the grammar and temporal semantics, and an AI coding assistant drafts the code, which the author reviews, tests, and corrects — making a full front-end, analyzer, VM, and scheduler feasible in one season. Idyl is pre-alpha: the core works, though syntax may still shift.

References

1. Dijkstra, E.W.: How Do We Tell Truths That Might Hurt? Manuscript EWD498 (1975). <https://www.cs.utexas.edu/~EWD/transcriptions/EWD04xx/EWD498.html>
2. Wang, G., Cook, P.R., Salazar, S.: ChucK: A Strongly Timed Computer Music Language. *Computer Music Journal* 39(4), 10–29 (2015)
3. McCartney, J.: Rethinking the Computer Music Language: SuperCollider. *Computer Music Journal* 26(4), 61–68 (2002)
4. Lazzarini, V. et al.: *Csound: A Sound and Music Computing System*. Springer (2016)
5. Csound GitHub site. <http://csound.github.io>
6. Puckette, M.: The Deadly Embrace Between Music Software and Its Users. In: *Proceedings of the Electroacoustic Music Studies Network Conference (EMS14): Electroacoustic Music Beyond Performance*. Berlin (2014)
7. Cont, A.: ANTESCOFO: Anticipatory Synchronization and Control of Interactive Parameters in Computer Music. In: *Proceedings of the International Computer Music Conference (ICMC)*, pp. 33–40. Belfast (2008)
8. Antescofo documentation. <https://antescofo-doc.ircam.fr/>
9. Giavitto, J.-L., Cont, A., Echeveste, J.: *Antescofo: A Not-So-Short Introduction to Version 0.x*. IRCAM, Paris (2015)
10. Echeveste, J., Cont, A., Giavitto, J.-L., Jacquemard, F.: Operational Semantics of a Domain Specific Language for Real Time Musician–Computer Interaction. *Discrete Event Dynamic Systems* 23(4), 343–383 (2013)
11. McLean, A.: Making Programming Languages to Dance to: Live Coding with Tidal. In: *Proceedings of the 2nd ACM SIGPLAN International Workshop on Functional Art, Music, Modeling & Design (FARM)*, pp. 63–70. ACM, Gothenburg (2014)
12. Orlarey, Y., Fober, D., Letz, S.: Faust: an Efficient Functional Approach to DSP Programming. In: *New Computational Paradigms for Computer Music*, pp. 65–96. Editions Delatour, Paris (2009)
13. Orlarey, Y., Fober, D., Letz, S.: Syntactical and Semantical Aspects of Faust. *Soft Computing* 8(9), 623–632 (2004)
14. Dannenberg, R.B., McAviney, P., Rubine, D.: Arctic: A Functional Language for Real-Time Systems. *Computer Music Journal* 10(4), 67–78 (1986)
15. Berry, G., Gonthier, G.: The Esterel Synchronous Programming Language: Design, Semantics, Implementation. *Science of Computer Programming* 19(2), 87–152 (1992)
16. Hudak, P., Courtney, A., Nilsson, H., Peterson, J.: Arrows, Robots, and Functional Reactive Programming. In: *Advanced Functional Programming*. LNCS, vol. 2638, pp. 159–187. Springer, Heidelberg (2003)
17. Nystrom, R.: *Crafting Interpreters*. Genever Benning (2021). <https://craftinginterpreters.com>
18. Idyl documentation, Chap. 12: Timing Accuracy (2026). https://johannphilippe.github.io/idyl/ch12_timing_accuracy.html
19. Rohrhuber, J., de Campo, A., Wieser, R.: Algorithms Today: Notes on Language Design for Just In Time Programming. In: *Proceedings of the International Computer Music Conference (ICMC)*, pp. 455–458. Barcelona (2005)